MLE-STAR OpenAI Port
Comprehensive Project Report

From Google ADK / Gemini to a university OpenAI-compatible API

A beginner-friendly, self-contained explanation

Date: June 15, 2026

Table of Contents

1. What this report is about

This document explains a software engineering project whose goal was to take a research prototype called MLE-STAR, originally built to run on Google's Agent Development Kit (ADK) and Gemini models, and make it run on a university-provided OpenAI-compatible large language model API. The report is written for a technically literate reader who knows programming and mathematics but has no prior knowledge of MLE-STAR, ADK, Kaggle competitions, or this specific repository.

By the end of this report you should understand: (a) what problem MLE-STAR tries to solve, (b) how the original system was organized, (c) why a port was necessary, (d) what the new implementation looks like, and (e) what results we obtained when we ran it for the first time.

2. The big picture: automating machine learning engineering

2.1 — Machine learning is still largely manual

In a typical machine learning project a human data scientist performs many steps: load the data, clean missing values, encode categorical variables, choose a model family, tune hyper-parameters, train the model, validate it on a held-out set, diagnose errors, iterate, and finally produce predictions on a test set. Each of these steps requires decisions, and many of them are guided by experience and trial-and-error.

2.2 — Competitions formalize the problem

Kaggle and similar platforms make this process concrete by providing: (1) a training file with known answers, (2) a test file without answers, and (3) an evaluation metric such as accuracy, RMSE, or log-loss. The participant submits a CSV file with predicted answers for the test set, and the platform scores the submission. The challenge is to build a pipeline that generalizes well from the training data to the test data.

2.3 — The dream: an autonomous ML engineer

The long-term research goal behind projects like MLE-STAR is to build an autonomous agent that receives a task description and data files, and then designs, implements, debugs, and submits a complete machine learning solution without human intervention. This is hard because the agent must both reason about the problem and write executable code, then observe the results of that code and improve.

2.4 — Large language models make this plausible

Modern large language models (LLMs) can write Python code, explain it, and modify it when given feedback. That opens the door to an agent loop: (1) ask the LLM for code, (2) run the code, (3) observe the score or error, and (4) ask the LLM to fix or improve it. MLE-STAR is exactly this kind of loop, wrapped in a multi-stage pipeline.

3. What MLE-STAR does

3.1 — Origin and paper

MLE-STAR stands for 'Machine Learning Engineering agent via Search and Targeted Refinement'. It is described in the Google Cloud AI Research paper of the same name. The system was designed to compete in Kaggle-style tabular machine learning competitions and reportedly won medals in a large fraction of the MLE-Bench-Lite datasets when powered by Google's Gemini models.

3.2 — High-level behavior

Given a task folder containing train.csv, test.csv, and a task_description.txt file, MLE-STAR:

1. Reads and summarizes the task.
2. Searches the web for recently effective models and example code for similar problems.
3. Generates one or more candidate solutions in Python.
4. Executes each candidate and records a validation score.
5. Selects the best candidate.
6. Refines the best candidate by identifying high-impact code blocks and trying improvements.
7. Optionally builds an ensemble that combines multiple good solutions.
8. Adds test-set loading logic and writes a submission.csv file.
9. Repeats and debugs whenever generated code fails.

3.3 — Why it is interesting

The interesting part is not just that an LLM writes code, but that the system has an outer loop of search, selection, refinement, and ensembling. It does not settle for the first script the model produces; it tries several, measures them, keeps the best, and improves it. This is much closer to how a human Kaggle grandmaster works than a single-shot code generator.

4. The original MLE-STAR architecture

4.1 — Built on Google's Agent Development Kit (ADK)

The original code was not a plain Python script. It was an ADK application. ADK is a framework from Google for building multi-agent systems. It provides primitives such as Agent, SequentialAgent, ParallelAgent, and LoopAgent, and it handles calling the LLM, passing state between agents, and invoking tools such as web search.

4.2 — File organization

The imported repository had the following structure:

```
mle-star-mcts-skrub/
├── machine_learning_engineering/
│   ├── __init__.py            # ADK setup, Vertex AI env vars
│   ├── agent.py               # root_agent and mle_pipeline_agent
│   ├── prompt.py              # system/frontdoor prompts
│   ├── shared_libraries/
│   │   ├── code_util.py       # run generated Python code
│   │   ├── common_util.py     # response parsing, seeding
│   │   ├── config.py          # default configuration
│   │   ├── debug_util.py      # retry/debug agent loops
│   │   └── check_leakage_util.py
│   └── sub_agents/
│       ├── initialization/    # task setup, search, first candidates
│       ├── refinement/        # ablation studies and improvements
│       ├── ensemble/          # combine multiple solutions
│       └── submission/        # final submission.csv generation
├── tasks/
│   └── california-housing-prices/
│       ├── train.csv
│       ├── test.csv
│       └── task_description.txt
├── tests/
├── deployment/
└── eval/
```

4.3 — The agent pipeline

At the top level, agent.py defined a SequentialAgent named mle_pipeline_agent that ran four sub-agents in order:

1. Initialization agent: summarized the task, searched for model candidates, generated code for each, executed them, and selected the best.
2. Refinement agent: ran ablation studies, found the most influential code blocks, generated improvement plans, and kept the best variant.
3. Ensemble agent: took the refined solutions, generated ensemble plans, executed them, and selected the best ensemble.

4. Submission agent: loaded the best solution, injected test-set prediction logic, executed it, and wrote submission.csv.

4.4 — Code execution and scoring

The execution engine lived in shared_libraries/code_util.py. Whenever an agent produced code, the framework wrote it to a .py file inside a task-specific workspace directory and ran it with subprocess. The generated script was required to print a line such as:

```
Final Validation Performance: 0.8473
```

code_util.py parsed that line, converted the number into a float, and stored it in a shared state object. Later agents used those scores to decide which solution to keep.

4.5 — State management

ADK provides a callback_context.state dictionary that persists across the whole pipeline. It contained configuration values, the task description, generated code, execution results, scores, and counters. Every agent read from and wrote to this state, allowing later stages to build on earlier ones.

4.6 — Prompts

Each sub-agent had its own prompt file. The prompts were carefully written to force the LLM to produce a single, self-contained Python script that loads data from ./input, avoids exit(), and prints the required performance line. This contract between the prompt and the execution engine is what made the loop possible.

5. Why we could not use the original system out of the box

5.1 — The LLM backend

The original code was tightly coupled to Google Cloud Vertex AI and the Gemini model family. It used google.auth to discover the active Google Cloud project and set environment variables such as GOOGLE_CLOUD_PROJECT, GOOGLE_CLOUD_LOCATION, and GOOGLE_GENAI_USE_VERTEXAI.

5.2 — The agent framework

All orchestration was expressed in ADK primitives: agents.Agent, agents.SequentialAgent, agents.ParallelAgent, agents.LoopAgent. The LLM client was hidden inside ADK. Replacing the model backend therefore required either staying inside ADK (which only supports Google models) or replacing ADK itself.

5.3 — Web search tool

The initialization and debug agents registered google_search as a tool. ADK knew how to call this tool and feed the results back to Gemini. There was no easy equivalent for a generic OpenAI-compatible endpoint provided by a university.

5.4 — Available infrastructure

The project owner had access to a university proxy that exposes an OpenAI-compatible chat completions endpoint and a limited budget of 25 EUR. There was no Google Cloud project, no Vertex AI access, and no Google AI Studio API key. Therefore the only viable path was to replace both the LLM client and the agent runtime.

6. The porting project: goals and constraints

6.1 — Primary goal

Make MLE-STAR generate, execute, and score a complete machine learning pipeline using the university OpenAI-compatible API, starting with the California Housing task.

6.2 — Secondary goals

- Preserve the original logic, prompts, and execution engine as much as possible.
- Keep the code modular so that refinement, ensemble, and web search can be re-enabled later.
- Maintain a clear state object that mimics the original ADK state.
- Produce a working submission.csv at the end of the run.

6.3 — Hard constraints

- Budget: approximately 25 EUR total for API usage.
- Model: meta-llama-3.1-8b-instruct, chosen for low cost.
- No web search available.
- No ADK, no Gemini, no Vertex AI.

6.4 — Consequence for scope

The full MLE-STAR pipeline generates multiple candidates, refines them, and ensembles them. That consumes a large number of tokens. With a 25 EUR budget and an 8-billion-parameter model, we could not afford the full pipeline. We therefore defined a Minimum Viable Product (MVP) that runs a single candidate, skips refinement and ensemble, and performs only minimal debugging.

7. The new architecture

7.1 — Overview

The new architecture keeps the same conceptual pipeline but replaces ADK with a small custom runner. The runner is responsible for calling the OpenAI-compatible API and orchestrating agents. Everything else — prompts, code execution, scoring, state keys — was kept as close to the original as possible.

7.2 — New file structure

```
machine_learning_engineering/
├── __init__.py             # OpenAI client + MODEL_NAME
├── agent.py                # run_pipeline() entry point
├── runner.py               # custom ADK replacement
├── prompt.py               # kept from original
├── shared_libraries/
│   ├── code_util.py        # adapted to plain state
│   ├── common_util.py      # OpenAI response parsing
│   ├── config.py           # MVP values
│   ├── debug_prompt.py     # debug/refine prompts
│   └── debug_util.py       # simplified retry/debug loop
└── sub_agents/
    ├── initialization/
    │   ├── agent.py        # simplified init pipeline
    │   └── prompt.py       # adjusted for no web search
    └── submission/
        ├── agent.py        # simplified submission pipeline
        └── prompt.py       # kept from original
```

7.3 — The runner

runner.py is the heart of the port. It defines AgentState (a plain dictionary that replaces ADK's callback_context.state), llm_call (a thin wrapper around client.chat.completions.create), and run_agent (the primitive that every other agent uses).

7.4 — The simplified pipeline

```
prepare task
   |
   v
summarize task
   |
   v
retrieve / propose model
   |
   v
generate and execute initial code
   |
   v
```

```
(if error) debug and retry
    |
    v
select best initial solution
    |
    v
generate and execute submission code
    |
    v
(if error) debug and retry
    |
    v
write submission.csv and final_state.json
```

8. Key design decisions

8.1 — One solution, one candidate

Original MLE-STAR generated num_solutions parallel branches and num_model_candidates candidates per branch. For the MVP we set both to 1. This eliminates parallel execution and reduces token consumption from several thousand to a few thousand per run.

8.2 — No refinement

Refinement uses ablation studies: it temporarily removes code blocks to measure their impact, then generates improvements for the most important block. This is valuable but expensive. We set inner_loop_round and outer_loop_round to 0, effectively disabling refinement in the MVP.

8.3 — No ensemble

Ensembling combines multiple refined solutions. It requires at least two good solutions and several ensemble plans. We set ensemble_loop_round to 0 and removed the ensemble agent from the pipeline.

8.4 — Minimal debug loop

The original debug loop could retry up to 10 times and perform 5 debug rounds. We reduced this to 1 retry and 1 debug round. The system still attempts to fix code once if it fails, but it gives up quickly to save tokens.

8.5 — No web search

Web search was one of the key features of the original STAR approach. Because no search tool was available through the university API, we instructed the LLM to propose a model from its internal knowledge. In practice the parsing of the model retrieval response failed with the small model, so the downstream code generation relied mainly on the task description.

8.6 — Prefer scikit-learn over PyTorch

The original prompts asked for PyTorch. The small Llama model produced neural network code with shape mismatches and other subtle bugs that were hard to debug. We adjusted the prompts to prefer scikit-learn for this tabular task, which produced robust RandomForest / GradientBoosting code and significantly improved success rate.

9. Implementation walkthrough

9.1 — Environment preparation

We created a new branch feature/issue-1.5-openai-port from the original import commit. We then:

1. Updated pyproject.toml to remove google-adk, google-genai, and google-cloud-aiplatform, and to add openai and python-dotenv.
2. Created a .gitignore file to exclude .env, .venv, __pycache__, and generated workspaces.
3. Updated .env.example with the university proxy variables OPENAI_API_BASE, OPENAI_API_KEY, and ROOT_AGENT_MODEL.
4. Removed ADK-dependent directories: tests/, deployment/, eval/, sub_agents/ensemble/, and sub_agents/refinement/.

9.2 — OpenAI client initialization

machine_learning_engineering/__init__.py was rewritten to load environment variables from .env, validate that OPENAI_API_KEY and OPENAI_API_BASE are present, and create a global OpenAI client. It also exposes MODEL_NAME, defaulting to meta-llama-3.1-8b-instruct.

```
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

API_KEY = os.environ.get("OPENAI_API_KEY")
API_BASE = os.environ.get("OPENAI_API_BASE")
MODEL_NAME = os.environ.get("ROOT_AGENT_MODEL", "meta-llama-3.1-8b-instruct")

client = OpenAI(api_key=API_KEY, base_url=API_BASE)
```

9.3 — The runner module

runner.py provides the primitives that replace ADK. AgentState is a dictionary subclass. llm_call invokes the chat completions endpoint. run_agent resolves instructions (strings or callables), supports optional before_model and after_model callbacks, and tracks token consumption.

9.4 — Adapting code_util.py

The original evaluate_code received an ADK callback_context. We changed it to receive (state, agent_name). All reads and writes now go through the plain AgentState dictionary. We also changed the subprocess invocation from ['python', ...] to [sys.executable, ...] so generated scripts run in the same virtual environment as the orchestrator, where pandas, scikit-learn, and torch are installed.

9.5 — Simplifying debug_util.py

The original debug_util built nested ADK LoopAgents. The new version exposes a single function run_and_debug that: (1) calls run_agent to generate and execute code, (2) checks whether execution succeeded and produced a score, and (3) if not, builds a BUG_REFINE_INSTR prompt from the error and asks the LLM to fix the code.

9.6 — Initializing the task

sub_agents/initialization/agent.py now has a function run_initialization_pipeline that: copies config into state, reads task_description.txt, creates the workspace directory, copies input data, summarizes the task, attempts model retrieval, generates the first code candidate, executes it, debugs if needed, and selects the best solution.

9.7 — Generating the submission

sub_agents/submission/agent.py creates the ensemble workspace (input and final directories), copies data there, and calls run_and_debug with the ADD_TEST_FINAL_INSTR prompt. This prompt instructs the LLM to take the best training script and add test-set loading logic that writes submission.csv into ./final.

9.8 — Entry point

machine_learning_engineering/agent.py defines run_pipeline, which initializes state, runs the initialization and submission pipelines, and saves the final state to final_state.json. When the file is run directly it prints the best score, the submission return code, and the total token consumption.

10. Prompt engineering

10.1 — The contract between prompt and executor

The most important design choice in the whole system is the contract between the LLM prompt and the execution engine. The prompt must make the LLM produce code that:

- Is a single, self-contained Python script.
- Loads data from ./input (train.csv and test.csv).
- Splits train.csv into train and validation sets for scoring.
- Does not use exit().
- Does not hide errors with try/except.
- Prints the exact line: 'Final Validation Performance: <number>'.
- For submission, writes ./final/submission.csv.

If any of these conditions is violated, code_util.py either refuses to run the script or cannot parse a score, and the agent fails.

10.2 — Adapting for the small model

meta-llama-3.1-8b-instruct is much smaller and weaker than Gemini-2.5-Pro. It tends to:

- Generate PyTorch code with shape mismatches.
- Use deprecated scikit-learn APIs such as mean_squared_error(..., squared=False).
- Ignore subtle instructions unless they are very explicit.
- Fail to follow strict JSON schemas.

To compensate, we added explicit reminders in multiple prompts: prefer scikit-learn, use root_mean_squared_error for RMSE in scikit-learn >= 1.7, and describe the exact column structure of the California Housing dataset.

10.3 — Example prompt excerpt

```
For California Housing: the train.csv has an ID column first,
then 8 feature columns, then the target column `median_house_value`.
Use `train_test_split` from sklearn on train.csv to create train
and validation sets.
```

11. Execution flow in detail

11.1 — Step-by-step trace

1. The user runs python -m machine_learning_engineering.agent.
2. __init__.py loads .env and creates the OpenAI client.
3. agent.py creates an AgentState and copies config into it.
4. run_initialization_pipeline is called.
5. prepare_task reads task_description.txt.
6. create_workspace copies train.csv, test.csv, and task_description.txt into workspace/<task>/1/input/.
7. A summarization agent asks the LLM to summarize the task.
8. A model retrieval agent asks the LLM to propose a model. (Parsing this response often fails with the small model, but the pipeline continues.)
9. A model evaluation agent asks the LLM to write the full training script.
10. get_code_from_response extracts the code block and writes it to init_code_1.py.
11. evaluate_code runs the script with subprocess in workspace/<task>/1/.
12. If the script prints 'Final Validation Performance: X', code_util parses X as the score.
13. If the script fails, run_and_debug builds a debug prompt and asks the LLM to fix the code.
14. rank_candidate_solutions and select_best_solution promote the best code to train_code_0_1.
15. run_submission_pipeline creates workspace/<task>/ensemble/, copies data there, and asks the LLM to add test prediction logic.
16. The final script writes ./final/submission.csv.
17. agent.py saves final_state.json and prints the results.

11.2 — State evolution

Important keys in the state dictionary include:

```
task_description        -> text read from task_description.txt
task_summary             -> LLM summary of the task
init_1_model_1          -> proposed model description (often empty in MVP)
init_code_1_1           -> first generated training script
init_code_exec_result_1_1 -> {returncode, stdout, stderr, score}
train_code_0_1          -> best selected training script
submission_code          -> final submission script
submission_code_exec_result -> result of running final script
best_score_1            -> best validation score
total_tokens_spent       -> cumulative tokens across all LLM calls
```

12. Results of the first successful run

12.1 — Output

```
=== Pipeline finished ===
Best score: 62724.541886
Submission return code: 0
Total tokens spent: 3352
```

12.2 — Interpretation

Best score is the RMSE (root mean squared error) on the internal validation split created from train.csv. A lower RMSE is better; 62,724 means that, on average, the predicted median house value is about $62,724 away from the true value. This is a plausible first result for an untuned model.

12.3 — Generated artifacts

The run created the following files:

- workspace/california-housing-prices/1/input/train.csv
- workspace/california-housing-prices/1/input/test.csv
- workspace/california-housing-prices/1/init_code_1.py
- workspace/california-housing-prices/1/train0.py
- workspace/california-housing-prices/ensemble/input/...
- workspace/california-housing-prices/ensemble/final_solution.py
- workspace/california-housing-prices/ensemble/final/submission.csv
- workspace/california-housing-prices/final_state.json

12.4 — Submission.csv

submission.csv contained 601 lines: 1 header line with the column name median_house_value and 600 prediction lines, one for each row in test.csv. The file was written successfully, which means the entire pipeline — from task description to submission file — executed end-to-end.

13. Problems encountered and how they were solved

13.1 — Missing Python dependencies in subprocess

Symptom: generated scripts failed with ModuleNotFoundError for pandas and torch. Cause: subprocess was invoking the system python instead of the virtual environment python. Fix: changed code_util.py to use sys.executable.

13.2 — Disk space exhaustion during installation

Symptom: pip install failed with 'No space left on device'. Cause: torch and CUDA packages are very large. Fix: installed CPU-only torch and cleaned pip cache and generated workspaces.

13.3 — PyTorch shape mismatches

Symptom: the generated neural network had input dimension 8 while the data provided 7 features. Cause: the small model miscounted columns and did not understand the ID column. Fix: switched prompts to prefer scikit-learn, which is more forgiving for tabular data.

13.4 — Deprecated scikit-learn API

Symptom: code failed with 'unexpected keyword argument squared'. Cause: scikit-learn 1.7 removed the squared parameter from mean_squared_error. Fix: added explicit instructions to use root_mean_squared_error.

13.5 — Model retrieval JSON parsing failed

Symptom: init_1_model_1 remained empty. Cause: the small model did not reliably output the exact JSON schema requested. Impact: the system still worked because the model evaluation prompt includes the full task description. Future work: simplify model retrieval to plain text or hardcode a default model for California Housing.

13.6 — Missing ensemble workspace

Symptom: submission failed with FileNotFoundError for ensemble/final_solution.py. Cause: the ensemble agent, which originally created that workspace, was removed. Fix: added workspace creation and data copying to the submission pipeline.

14. Current limitations

14.1 — No web search

The STAR part of MLE-STAR is currently disabled. The LLM relies entirely on its pre-trained knowledge, which may be outdated or wrong for specialized tasks.

14.2 — No refinement

The system accepts the first working solution. It does not perform ablation studies or targeted improvements, so the score is likely far from what a human or the full MLE-STAR could achieve.

14.3 — No ensemble

Only one solution is produced. Ensembling multiple diverse models is one of the most effective ways to improve competition scores, and it is not available in the MVP.

14.4 — Single-task focus

Prompts were tuned for California Housing. Other tasks may require similar tuning to handle different column structures, target types, or metrics.

14.5 — Small model limitations

meta-llama-3.1-8b-instruct is cost-effective but produces more bugs and ignores instructions more often than larger models. The debug loop compensates partially, but it cannot fix every error.

14.6 — Token budget

Each run consumes 3,000–10,000 tokens. While a single run is cheap, iterating many times or re-enabling refinement and ensemble could exhaust the 25 EUR budget quickly.

15. Possible next steps

15.1 — Optimize model retrieval

Either remove the model retrieval call to save tokens, or replace the JSON schema with plain-text extraction. A reliable model retrieval step would give the evaluation agent better guidance.

15.2 — Re-enable refinement

Once budget allows, re-implement the ablation and refinement agents. This is the part of MLE-STAR most likely to substantially improve the validation score.

15.3 — Re-enable ensemble

After refinement produces two or more good solutions, add the ensemble agent to combine them.

15.4 — Add a web search tool

Implement a search wrapper using DuckDuckGo (free but unofficial), SerpAPI, or Bing Search API. Connect it to the runner's tool mechanism so the model retrieval agent can query the web.

15.5 — Experiment with larger models

If budget permits, test larger models available through the university proxy. Larger models usually follow instructions better, produce fewer bugs, and may make refinement and ensemble worthwhile.

15.6 — Generalize to other tasks

Make prompts more task-agnostic by detecting the target column, metric, and feature types from the task description or by inspecting the CSV files programmatically.

15.7 — Add cost tracking

Extend the runner to estimate cost per run based on token counts and model pricing. This would help manage the 25 EUR budget.

16. Glossary

Term	Meaning
ADK	Google's Agent Development Kit, a framework for building multi-agent LLM applications.
Agent	A component that performs a specific role, often backed by an LLM.
API	Application Programming Interface; here, the HTTP endpoint for the LLM.
Ensemble	Combining predictions from multiple models to improve performance.
LLM	Large Language Model, a neural network trained to generate text and code.
MCTS	Monte Carlo Tree Search, a planning algorithm (future goal, not in MVP).
ML pipeline	The sequence of steps from raw data to trained model and predictions.
MVP	Minimum Viable Product, the smallest version that demonstrates end-to-end functionality.
OpenAI-compatible	An API that follows the same request/response format as OpenAI.
Prompt	The text instruction given to an LLM.
RMSE	Root Mean Squared Error, a common regression evaluation metric.
skrub	A Python library for dirty tabular data (future goal, not used in MVP).
State	A shared dictionary that persists data across agents during a run.
Submission.csv	The file of predictions uploaded to a competition platform.
Tabular data	Data organized in rows and columns, like a spreadsheet or CSV.
Token	A unit of text processed by an LLM; usage determines API cost.
Validation set	A portion of training data held out to estimate model performance.
Vertex AI	Google Cloud's machine learning platform.

This concludes the report. The project demonstrated that MLE-STAR's core loop — LLM code generation, execution, scoring, and submission — can be ported to a generic OpenAI-compatible API, even with a small model and a limited budget.